Arquitetura e metodologia do dashboard de Heliópolis

Guia de estudo do fluxo de dados e dos cálculos implementados

Este documento explica a implementação atual do dashboard: como os dados de clima e de internações são obtidos, tratados e utilizados; quais cálculos produzem cada gráfico; e como os arquivos Python se comunicam. O período analisado é 2021–2025, definido como os cinco anos completos anteriores a 2026.

Engenharia de software e arquitetura

O projeto é uma aplicação monolítica modular em Python: a interface Streamlit, a coleta e as análises executam no mesmo processo, com responsabilidades separadas em arquivos. O navegador apresenta o painel e envia as interações; os módulos Python processam os dados com Pandas. A API REST consumida é externa, do Open-Meteo. Os dados de saúde vêm de uma tabela local de internações do SIH/SUS, preparada fora do dashboard. O projeto não expõe uma API REST própria nem utiliza banco de dados.

Requisitos funcionais implementados

Código	Requisito	Comportamento
RF01	Obter dados	Ler o JSON ERA5 local ou consultar o Open-Meteo quando ele não existir.
RF02	Tratar a série	Validar campos e unidades, organizar a grade horária, remover duplicatas e preservar lacunas.
RF03	Explorar o período	Filtrar 2021–2025 por ano e ajustar o limiar analítico entre 25 e 40 °C.
RF04	Analisar o calor	Calcular intensidade, frequência, perfil horário, noites, umidade e sequências de calor.
RF05	Comparar calor e internações	Comparar as internações por doenças do coração e respiratórias nos dias quentes e em cada faixa de temperatura com o esperado, com intervalo de 95%.
RF06	Apresentar resultados	Exibir gráficos, cartões, critérios e limites; gerar relatório e auditorias pelo terminal.

Requisitos não funcionais e limites operacionais

Os requisitos funcionais descrevem ações observáveis, como filtrar anos e calcular episódios. Os requisitos não funcionais definem condições de qualidade e operação dessas ações. A tabela registra os critérios adotados no código, com os limites que ainda não foram avaliados ou implementados.

Código	Atributo	Critério atual
RNF01	Qualidade	Exigir 90% de cobertura diária e noturna de temperatura; não inventar nem interpolar lacunas.
RNF02	Rastreabilidade	Preservar a resposta JSON de origem, os parâmetros, o período, o fuso e os metadados da coleta.
RNF03	Eficiência	Reutilizar sessão e cache para evitar consultas repetidas. Não há meta de latência medida ou SLA.
RNF04	Falhas explícitas	Tratar falhas previstas e mostrar indisponibilidade. Não há retentativa automática contínua.
RNF05	Manutenção	Separar módulos, comentar regras e manter testes. A aplicação depende diretamente de Streamlit e Pandas.
RNF06	Clareza	Usar uma página, unidades e ausência explícitas; distinguir contexto regional de medições locais e de impactos na saúde.

Escopo ainda não implementado

Etapa atual do estudo: relacionar as condições regionais estimadas por ERA5 a dados de saúde. Já está integrada uma tabela de internações do SIH/SUS da cidade de São Paulo inteira (não do bairro). Dados de qualidade de vida e do próprio território de Heliópolis ainda não estão integrados à aplicação.

Os requisitos acima descrevem o comportamento atual, não uma certificação de qualidade. A execução é local; autenticação, controle de acesso para dados pessoais e operação pública com alta disponibilidade não foram implementados.

Como verificar os requisitos

Os testes automatizados verificam filtros, cobertura, lacunas, fronteiras de datas e tratamento da resposta da API. A inspeção dos metadados permite rastrear a origem dos dados. Sessão e cache reduzem trabalho repetido, mas não constituem teste de carga ou garantia de tempo de resposta. Falhas previstas recebem tratamento; indisponibilidade externa, falhas inesperadas e alterações no formato das fontes ainda podem interromper a análise.

Arquitetura das requisições e do processamento

A aplicação consome a API REST do Open-Meteo ou lê o JSON dessa mesma fonte salvo localmente. Os dados de saúde entram por uma tabela local, gerada por um pipeline offline que roda no terminal. As setas representam as trocas de dados e a sequência das responsabilidades; não representam microsserviços independentes.

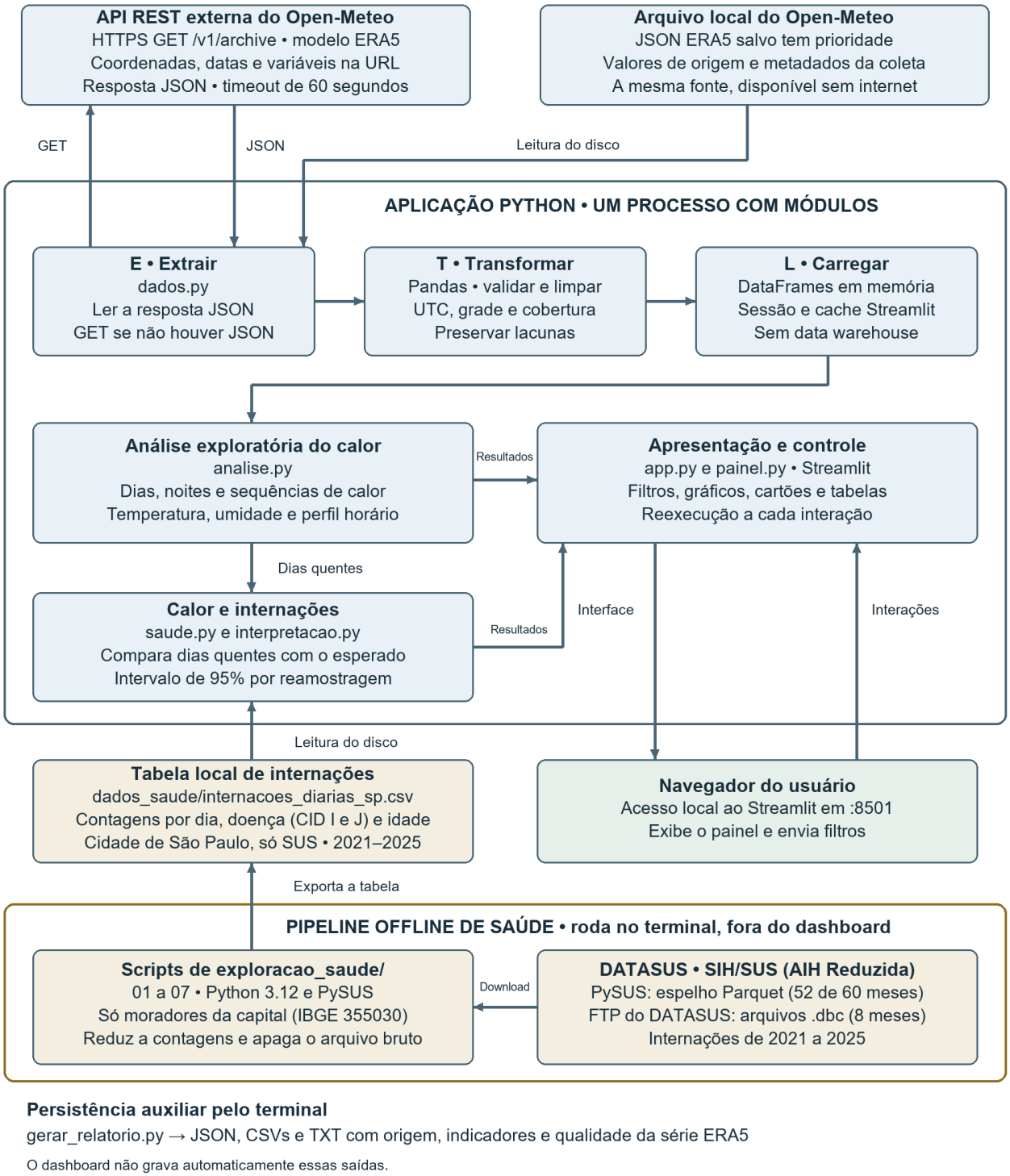


Figura de arquitetura Requisições externas, ETL em memória, tabela de internações e pipeline offline de saúde

Na carga inicial, o sistema prioriza o JSON salvo. Se ele não existir, o servidor Python envia uma requisição HTTPS GET ao endpoint /v1/archive do Open-Meteo, com coordenadas, período, modelo ERA5 e variáveis na URL; a resposta retorna em JSON. O Open-Meteo é a única fonte meteorológica consumida pela aplicação. O dashboard não consulta o DATASUS: a tabela de internações é lida do disco.

O fluxo segue um ETL em memória: extrair o JSON local ou recebido da API; transformar campos, unidades, datas, cobertura e duplicatas; carregar DataFrames tratados para os cálculos e a visualização. A carga não ocorre em um data warehouse. Os scripts auxiliares geram arquivos persistentes quando executados; o dashboard reutiliza sessão e cache. As internações seguem um caminho separado: scripts em exploracao_saude/ baixam o SIH/SUS, reduzem os arquivos a contagens e exportam dados_saude/internacoes_diarias_sp.csv.

Os requisitos acima descrevem o comportamento atual, não uma certificação de qualidade. A execução é local; autenticação, controle de acesso para dados pessoais e operação pública com alta disponibilidade não foram implementados.

Contexto do estudo e explicação do ERA5

Objetivo e alcance atual

O projeto utiliza Ciência de Dados para estudar as condições de calor relacionadas à saúde e à qualidade de vida dos moradores de Heliópolis. A etapa atual combina a série ERA5 disponibilizada pelo Open-Meteo com uma tabela de internações hospitalares do SIH/SUS da cidade de São Paulo; a investigação de associações com qualidade de vida e com o território de Heliópolis depende de dados compatíveis ainda não obtidos.

Como não há medições dentro da comunidade e os dados de saúde cobrem a cidade inteira (apenas internações pelo SUS), a versão documentada descreve condições regionais de calor e as compara com as internações da cidade. Ela não mede temperaturas dentro das moradias, sintomas, qualidade do sono ou impactos individuais.

O que é ERA5

ERA5 é a quinta geração de reanálise atmosférica do ECMWF, produzida para o Serviço de Mudanças Climáticas Copernicus. Uma reanálise combina observações históricas, como medições de estações e satélites, com um modelo físico e técnicas de assimilação de dados para estimar o estado passado da atmosfera. Oferece séries horárias globais desde 1940. [1, 2]

Isso permite contextualizar as condições meteorológicas de uma região ao longo do tempo. Os valores são estimativas de reanálise, e não leituras de um termômetro instalado no ponto consultado. A resolução nativa do ERA5 é de aproximadamente 31 km; produtos distribuídos em grade regular usam 0,25°. O Open-Meteo descreve essa grade como cerca de 25 km. [1, 2, 3]

O projeto acessa ERA5 pelo Open-Meteo com models=era5. Open-Meteo é a interface de acesso; ERA5 é a base meteorológica. A API pode aplicar ajustes estatísticos de altitude por padrão: o código não define elevation=nan para desativá-los. Isso não transforma a série em uma medição do microclima de Heliópolis. [3]

A série regional ajuda a explorar intensidade, frequência e persistência do calor. Para relacioná-la à experiência dos moradores de Heliópolis, ainda serão necessários dados de saúde e qualidade de vida compatíveis no tempo e no território do bairro.

1 Fluxograma da sequência dos dados

O fluxo usa uma única fonte meteorológica: ERA5 pelo Open-Meteo. A mesma série pode ser obtida pela API ou por seu JSON salvo. Depois da validação e do recorte, ela alimenta todos os gráficos e indicadores. O módulo saude.py cruza os dias quentes com uma tabela local de internações, que não vem da internet em tempo de execução.

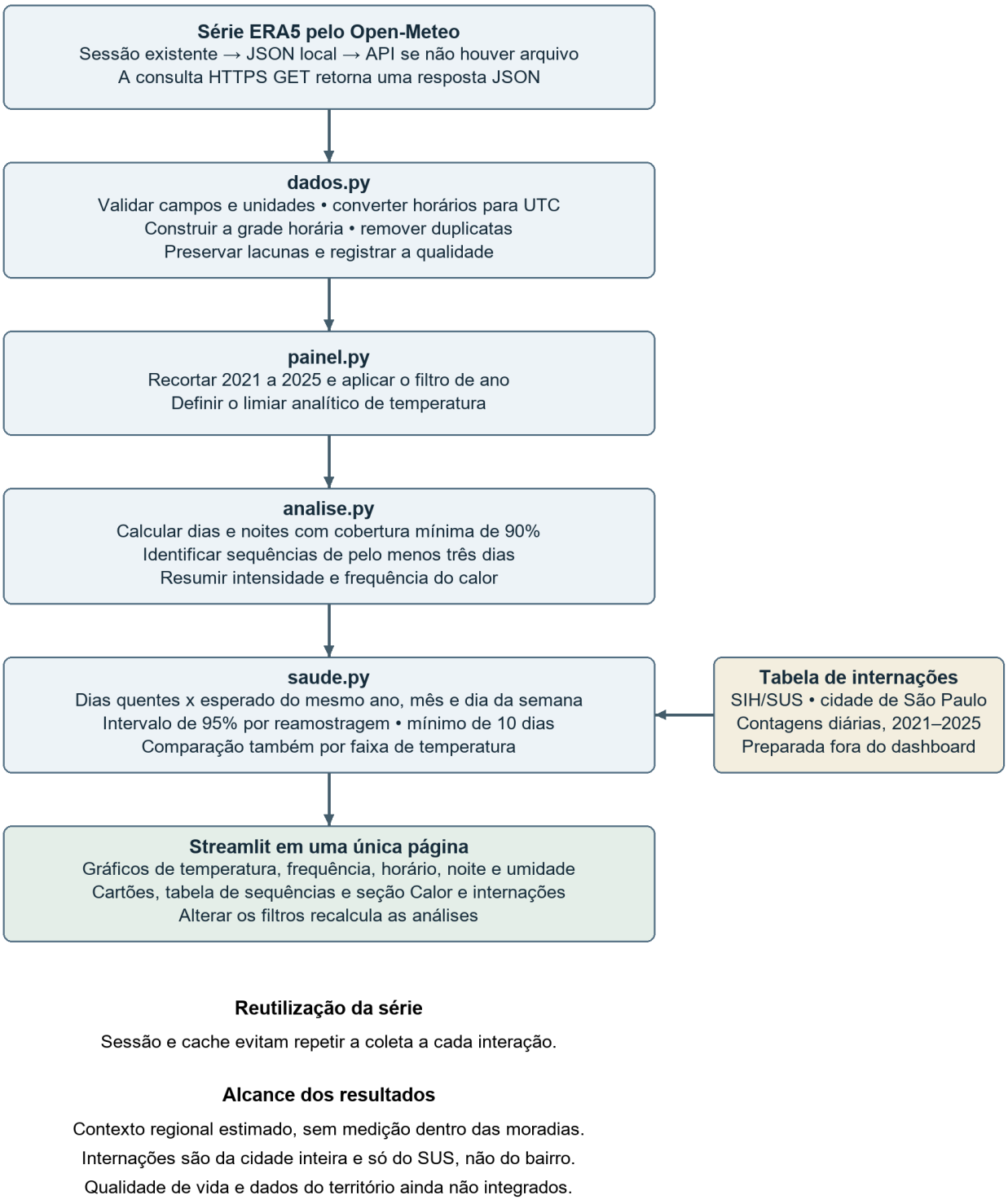


Figura 1 Sequência implementada de entrada, tratamento, análise, comparação com internações e apresentação dos dados

Os módulos se comunicam por chamadas de funções e objetos Python em memória. Não há banco de dados, API interna ou serviço de processamento separado.

2 Coleta e tratamento da série ERA5

Prioridade de carregamento

Ao abrir a aplicação, iniciar_base verifica primeiro se existe um conjunto em st.session_state. Se existir, ele é reutilizado. Na primeira carga, procura dados_locais/heliopolis_open_meteo.json. Se o arquivo existir, ele é lido e validado. Somente se não existir, carregar_online consulta o Open-Meteo. Um arquivo inválido gera indisponibilidade; não há substituição automática por dados artificiais.

A consulta usa urllib, da biblioteca padrão, com o endpoint https://archive-api.open-meteo.com/v1/archive. Solicita latitude −23,615 e longitude −46,590, modelo ERA5, timestamps Unix e fuso UTC. O ponto é aproximado e não corresponde a um centroide territorial oficialmente delimitado pelo projeto.

Variável solicitada	Nome interno	Unidade
Temperatura do ar a 2 m	temperatura_c	°C
Umidade relativa a 2 m	umidade_pct	%
Vento a 10 m	vento_ms	m/s
Radiação solar	radiacao_wm2	W/m²
Precipitação	precipitacao_mm	mm

A resposta JSON contém listas horárias e unidades. interpretar_api verifica os campos, converte timestamps e cria um DataFrame indexado por horário. A interface atual utiliza principalmente temperatura e umidade; vento, radiação e precipitação permanecem no processamento auxiliar.

Limpeza e cobertura

limpar_horarios ordena a série, remove horários duplicados mantendo a primeira ocorrência e reindexa em uma grade com todas as horas esperadas. Em 2021–2025 são 43.824 horas e 1.826 dias. Registros ausentes e valores inválidos viram NaN; não são interpolados nem substituídos por zero.

Variável	Intervalo de consistência aceito
Temperatura	−90 a 65 °C
Umidade	0 a 100%
Vento	0 a 150 m/s
Radiação	0 a 1.600 W/m²
Precipitação horária	0 a 1.000 mm

Esses limites amplos verificam consistência básica. Não são limites de saúde e não substituem controle completo de qualidade. Para cada variável, são registrados horas esperadas, válidas, ausentes ou inválidas, valores fora dos limites e percentual de cobertura.

montar_conjunto reúne horários, qualidade, número de duplicatas, metadados e resposta de origem. Os metadados preservam modelo, período, coordenadas, data de coleta, fuso e URL da consulta, permitindo rastrear a proveniência.

3 Cálculos diários e gráficos de temperatura

Regra de cobertura diária

Se n é a quantidade de registros válidos e H a quantidade de horas esperadas no dia, a cobertura é calculada pela expressão abaixo. O dia é válido com pelo menos 90% das horas de temperatura.

$$Cobertura\ diária\ (\%)\ =\ 100\ \times\ n\ /\ H$$

Em um dia de 24 horas, são necessários pelo menos 22 registros válidos. Um dia inválido fica com temperatura agregada e classificação de calor ausentes; ele não é interpretado como um dia ameno. Os agrupamentos usam o fuso America/Sao_Paulo.

Temperatura ao longo do tempo

Para cada dia válido, a média é a soma das temperaturas válidas dividida pela quantidade de registros; a máxima e a mínima são os extremos dessas amostras horárias.

$$Temperatura\ média\ diária\ =\ soma\ das\ temperaturas\ válidas\ /\ n$$

Até 370 dias, o gráfico exibe medidas diárias. Em períodos maiores, exibe a média mensal de cada medida diária. A linha mensal de máxima é a média das máximas dos dias válidos daquele mês, não o maior valor ocorrido no mês. O mesmo vale para média e mínima. As agregações mensais não impõem um segundo limiar mínimo de dias válidos.

Essa visualização permite explorar intensidade e sazonalidade. Máximas horárias podem não captar picos ocorridos entre as amostras. Cartões e episódios continuam usando medidas diárias, sem a suavização mensal do gráfico.

Frequência de dias quentes

Um dia válido é classificado como quente quando sua máxima é maior ou igual ao limiar L. O padrão é 30 °C, ajustável de 25 a 40 °C em passos de 0,5 °C. Esse é um critério analítico, não um limite clínico.

$$Frequência\ anual\ (\%)\ =\ 100\ \times\ dias\ válidos\ com\ máxima\ \geq\ L\ /\ dias\ válidos$$

Exemplo: 70 dias acima do limiar entre 350 dias válidos representam 20%. O denominador não é automaticamente 365 ou 366. Não há um limiar adicional de cobertura anual; a frequência descreve os dias disponíveis e não estabelece tendência climática com apenas cinco anos.

Calor por horário

A série horária é convertida ao horário local e agrupada de 0h a 23h. Para cada hora, calcula-se a média dos registros válidos daquela hora ao longo do período. O ponto das 14h representa a média de todos os registros válidos às 14h.

Esse gráfico inclui registros válidos de dias que eventualmente não atingem 90% de cobertura diária. Não é filtrado pelo limiar L. Ele descreve o perfil térmico médio ao longo da rotina, sem representar exposição pessoal medida.

4 Noites umidade e persistência do calor

Temperatura à noite

Cada noite começa às 22h e termina às 05h59 do dia seguinte. Os registros da madrugada são associados à data de início da noite. A janela completa deve estar dentro do período consultado e ter pelo menos 90% de cobertura.

Uma noite usual tem oito registros horários. Como sete correspondem a 87,5%, são necessários os oito para atingir o critério. O código também considera janelas de sete ou nove horas em transições históricas de horário de verão.

$$Média\ noturna\ =\ soma\ das\ temperaturas\ noturnas\ válidas\ /\ registros\ válidos$$

Se a tabela tem mais de 370 linhas, a visualização apresenta a média mensal das médias noturnas, agrupando pela data de início da noite. Caso contrário, mostra cada noite. Noites parciais nas bordas são excluídas. A análise descreve o ambiente durante o descanso, sem medir sono ou temperatura dentro das casas.

Calor e umidade

Cada ponto da dispersão corresponde às médias de temperatura e umidade de um dia. O dia precisa ser válido para temperatura e ter cobertura de pelo menos 90% para umidade. A verificação é independente por variável: as horas efetivamente usadas nas duas médias podem diferir.

O eixo horizontal recebe a temperatura média diária; o vertical recebe a umidade média diária. Todos os dias elegíveis aparecem, inclusive aqueles abaixo do limiar. Não há cálculo de Pearson, regressão, sensação térmica, UTCI ou índice fisiológico nessa visualização.

Sequências de calor persistente

episodios_quentes percorre os dias em ordem cronológica e identifica sequências de pelo menos três dias consecutivos com máxima ≥ L. Um dia abaixo do limiar, inválido ou ausente interrompe a sequência. Um episódio longo conta uma vez, e não como várias janelas sobrepostas de três dias.

Medida do episódio	Como é calculada
Início e fim	Primeiro e último dia da sequência
Duração	Número de dias consecutivos
Maior temperatura	Maior máxima diária da sequência
Média das máximas	Média das máximas diárias da sequência
Borda do recorte	Indica contato com o início ou o fim da consulta

Ao filtrar um ano, episódios que atravessam a virada podem ser cortados. A marcação de borda indica que podem continuar fora do recorte. A definição de três dias é exploratória e não corresponde automaticamente a uma onda de calor oficial.

5 Cartões e análises auxiliares

Cartão	Cálculo implementado
Maior temperatura horária	Maior máxima diária entre os dias válidos
Dias com máxima ≥ limiar	Soma dos dias válidos classificados como quentes
Sequências de calor	Quantidade de episódios com três dias ou mais
Temperatura média noturna	Média das médias de todas as noites válidas

No cartão noturno, cada noite válida tem o mesmo peso. Quando não há suporte suficiente, resultados ausentes são apresentados como travessão, sem confundir ausência com zero.

Cálculos auxiliares fora da tela atual

O módulo também calcula horas observadas acima do limiar, médias diárias de vento e radiação e chuva diária. A chuva exige 100% de cobertura antes de somar os valores; as demais médias auxiliares exigem 90%. Nenhuma lacuna é extrapolada. Esses resultados podem ser consultados nos arquivos gerados pelo terminal.

6 Calor e internações (SIH/SUS)

De onde vêm os dados de saúde

As internações vêm do Sistema de Informações Hospitalares do SUS (SIH/SUS), arquivos AIH Reduzida (RD) de São Paulo, publicados pelo DATASUS. O dashboard não consulta o DATASUS: lê a tabela pronta dados_saude/internacoes_diarias_sp.csv, que foi gerada antes por um pipeline separado. Um arquivo .json ao lado dela registra a fonte, os filtros, a data de geração e o número de linhas.

Coluna da tabela	Conteúdo
data	Dia da internação, de 01/01/2021 a 31/12/2025
grupo_cid	I00-I99 (aparelho circulatório) ou J00-J99 (aparelho respiratório)
faixa_etaria	0-14, 15-64, 65+ ou sem idade
n_internacoes	Número de internações naquele dia, grupo e faixa; dias sem internação aparecem com zero

Entram apenas moradores da cidade de São Paulo (código IBGE 355030) com AIH normal. A tabela não contém CEP, idade exata, hospital nem qualquer dado individual. Ela descreve a cidade inteira e somente as internações pelo SUS; não identifica moradores de Heliópolis.

Pipeline offline em exploracao_saude/

Os scripts da pasta exploracao_saude/ rodam no terminal, com ambiente próprio (Python 3.12 e PySUS), e não fazem parte da aplicação Streamlit. O catálogo do PySUS tem 52 dos 60 meses de 2021–2025; os 8 meses ausentes foram obtidos no FTP do DATASUS, em arquivos .dbc. Os arquivos de competências até 2026-08 foram usados para completar as internações de 2025, que chegam com atraso.

Script	O que faz
01 e 01b	Listam os arquivos disponíveis no catálogo do PySUS e no FTP
02 e 03	Exploram um mês: colunas, perfil e contagem diária
04 e 05	Baixam mês a mês, reduzem cada arquivo a contagens, apagam o arquivo bruto e montam as contagens mensal e diária
06 e 08	Cruzam as internações com a temperatura e conferem o método do painel com regressão de Poisson, em vários limiares
07	Exporta a tabela dados_saude/internacoes_diarias_sp.csv que o dashboard lê

O relatório exploracao_saude/RELATORIO.md resume o que foi feito e conclui que os dados do SIH permitem uma série diária da cidade, mas não chegam ao bairro de Heliópolis.

Como a comparação é calculada

Dias de calor e dias comuns diferem também por mês do ano, ano e dia da semana, que afetam as internações independentemente do clima. Para reduzir esse efeito, cada dia quente é comparado com o valor esperado: a média das internações dos dias não quentes do mesmo ano, mês e dia da semana. Os dias quentes ficam fora dessa média. Um dia quente só entra se houver pelo menos dois dias de referência.

$$Diferença\ (\%) = 100 \times (internações\ reais\ /\ internações\ esperadas\ nos\ dias\ quentes - 1)$$

O intervalo de 95% vem de sortear, com reposição, os dias quentes 2.000 vezes (reamostragem). Se o intervalo inclui o zero, o painel não mostra diferença clara. Com menos de 10 dias quentes ou menos de 10 dias de comparação, o resultado não é calculado e a tela orienta o usuário.

Gráfico da seção	O que compara
Internações nos dias quentes	Dias com máxima ≥ limiar do painel, por grupo de doença e por idade (todas as idades e 65 anos ou mais)
Internações por faixa de temperatura	Cada faixa de temperatura máxima (abaixo de 20, 20 a 24, 24 a 27, 27 a 30 e 30 °C ou mais) contra os dias fora da faixa, para todas as idades; não depende do limiar

Os gráficos de barras desta seção usam Altair, instalado junto com o Streamlit, para deixar os nomes do eixo na horizontal. saude.py faz as contas e interpretacao.py escreve as leituras em linguagem simples (leitura_saude e leitura_faixas), sem desenhar gráficos.

Limites de interpretação

É uma comparação descritiva e mostra associação, não causa. Os dados são da cidade inteira e só do SUS, enquanto o calor é uma estimativa regional. Diferenças pequenas podem ser acaso, e faixas muito frias ou muito quentes têm menos dias e resultados mais incertos. O texto da tela traz esses avisos junto de cada resultado.

7 Responsabilidades dos arquivos principais

app.py e painel.py

app.py é o ponto de entrada do comando streamlit run app.py. Ele importa main de painel.py e executa a função. Não contém regras de cálculo ou coleta.

painel.py coordena a aplicação. iniciar_base obtém a base; recortar_base aplica o ano selecionado; main chama indicadores_diarios, indicadores_noturnos e episodios_quentes. mostrar_exposicao desenha os cartões e gráficos de evolução e frequência; mostrar_rotina apresenta horário, noite e umidade; mostrar_persistencia exibe os episódios; mostrar_saude desenha a seção Calor e internações, com barras_agrupadas (Altair) para os dois gráficos de barras.

A interface chama numero de apoio.py para formatar os valores apresentados. Parte das agregações visuais, como médias mensais e perfil horário, ainda está em painel.py. A separação entre apresentação e cálculo é, portanto, parcial.

dados.py

Centraliza constantes do estudo, validação de períodos, consulta HTTP, leitura e exportação JSON, grade horária e limpeza da série ERA5. Não depende do Streamlit. ErroDados fornece mensagens que podem ser tratadas pela interface ou pelo terminal.

Chave do conjunto	Conteúdo
horarios	DataFrame com a série horária tratada
qualidade	DataFrame com cobertura e inconsistências
duplicatas	Número de horários duplicados removidos
metadados	Dicionário com origem e parâmetros da coleta
resposta_api	Dicionário com a resposta de origem do período

analise.py

Recebe tabelas do Pandas e devolve resultados analíticos sem exigir interface. indicadores_diarios agrega dias e verifica cobertura; indicadores_noturnos organiza janelas de descanso; episodios_quentes identifica sequências; resumo_anual calcula contagens, médias e frequências. gerar_relatorio monta o texto auxiliar com critérios, indicadores e limitações.

saude.py e interpretacao.py

saude.py lê a tabela de internações (carregar_internacoes), escolhe grupo de doença e idade (serie_diaria) e compara os dias quentes e cada faixa de temperatura com o esperado (comparar_todos e comparar_por_faixas). Recebe a tabela diária de analise.py e não desenha gráficos nem escreve textos.

interpretacao.py reúne os textos explicativos do dashboard: fontes, glossário, conclusões e as leituras automáticas que citam os números do recorte, como leitura_saude e leitura_faixas. Não faz contas estatísticas.

8 Comunicação execução e persistência

apoio.py e arquitetura_projeto.py

apoio.py contém apenas numero, responsável por casas decimais, separadores em português e ausência exibida como travessão.

arquitetura_projeto.py centraliza o texto dos requisitos, a descrição arquitetural e o caminho da imagem usados no dashboard e neste documento. assets/arquitetura_projeto.png é a imagem compartilhada. Os geradores em documentacao/ preparam os artefatos e não são executados pelo dashboard; suas dependências de edição de documentos não fazem parte da aplicação Streamlit.

gerar_relatorio.py

É uma entrada alternativa pelo terminal. Usa dados.py e analise.py para gerar um JSON de origem, CSVs horário, diário, noturno, episódios, resumo anual e qualidade, além de parametros.json e relatorio_inicial.txt. O argumento --arquivo permite repetir a análise offline; --inicio e --fim delimitam o período; --saida define o destino.

Seus critérios são fixos em 30 °C, três dias e 90% de cobertura. O script não recebe automaticamente o limiar selecionado na sessão do dashboard. A execução pode substituir os arquivos de saída com os mesmos nomes.

Reexecução e cache

Ao alterar um filtro, o Streamlit executa novamente a página. st.session_state conserva o conjunto ERA5 da sessão. carregar_online usa cache de 24 horas e até seis entradas.

O ano altera todas as análises. O limiar altera a classificação dos dias, frequência anual e episódios. Ele também define quais dias entram na comparação com internações nos dias quentes, mas não altera as temperaturas, os gráficos de evolução, perfil horário, noite e dispersão, nem o gráfico por faixa de temperatura.

A consulta online do painel não grava automaticamente o JSON em disco. Essa persistência é feita pelo script de relatório. Um JSON alterado no disco também não substitui por si só o conjunto já carregado na sessão. Uma falha na primeira carga não provoca tentativas repetidas a cada filtro na mesma sessão.

Objetos que atravessam os módulos

A comunicação usa funções importadas, DataFrames, dicionários e datas. Não existe uma API interna. O caminho principal é app → painel → dados e analise; para a seção de saúde, painel chama saude e interpretacao, e saude recebe a tabela diária de analise. painel também importa textos de arquitetura_projeto e a formatação de apoio. gerar_relatorio chama dados e analise sem iniciar Streamlit.

9 Armazenamento testes e limites de interpretação

Arquivo ou pasta	Função
dados_locais/heliopolis_open_meteo.json	Base ERA5 de entrada preferencial do painel
Outros CSVs e TXT em dados_locais/	Resultados auxiliares gerados pelo terminal
dados_saude/	Tabela diária de internações do SIH/SUS (CSV) e arquivo .json com a origem; é lida pelo painel
exploracao_saude/	Scripts, saídas e relatório do pipeline offline que gera a tabela de saúde; fora do dashboard
requirements.txt	Dependências diretas da aplicação: Streamlit e Pandas
.gitignore	Exclui dados locais, ambiente virtual, caches e segredos do Git
.venv/ e __pycache__/	Ambiente de execução e cache de bytecode Python

O dashboard lê o JSON ERA5 ou consulta a API quando ele não existe, recalculando os resultados. Os CSVs diários exportados pelo terminal são saídas auxiliares e não alimentam o painel; a tabela de internações em dados_saude/ é a exceção, porque o painel a lê. Imagens do dashboard servem para conferência visual e não alimentam a análise.

Testes automatizados

Arquivo em tests/	Responsabilidade
test_relatorio.py	Limpeza, cobertura, datas, noites, episódios, API e relatório
test_interface.py	Abertura, filtros e ausência de controles técnicos removidos
test_saude.py	Comparação com o esperado, intervalo, dias inválidos, faixas de temperatura, tabela sem dados pessoais e seção no painel
test_interpretacao.py	Textos dinâmicos, perfil horário, correlação, glossário e conclusões

Os testes utilizam dados sintéticos exclusivamente para validar o comportamento do código. Eles não são inseridos na base apresentada aos moradores ou pesquisadores.

O que os resultados permitem afirmar

A implementação permite descrever o contexto regional de calor e levantar perguntas sobre intensidade, frequência, persistência e condições noturnas. Os critérios de cobertura e a preservação de lacunas tornam explícito o suporte de dados de cada resultado.

Ela não calcula efeitos clínicos, causalidade, desconforto individual, ilha de calor dentro de Heliópolis ou tendências climáticas robustas com cinco anos. O limiar de temperatura não é uma classificação nacional universal de conforto. A grade regional de reanálise não permite quantificar diretamente o excesso térmico dentro do bairro.

A seção de internações mostra associação entre calor regional e internações da cidade inteira pelo SUS; não identifica efeito no bairro nem causa. Para cumprir integralmente o objetivo de impactos na saúde e na qualidade de vida em Heliópolis, será necessário integrar desfechos compatíveis com a exposição ambiental

estimada no tempo e no território do bairro, com definições e cobertura conhecidas. Essa etapa exige novos dados e regras analíticas, não apenas novos gráficos.

10 Fontes e rastreabilidade da documentação

A descrição dos módulos e cálculos foi conferida no código local vigente em 24 de outubro de 2026. As referências externas abaixo fundamentam a explicação do ERA5 e de sua disponibilização pelo Open-Meteo. Elas não substituem a verificação dos parâmetros efetivamente usados pelo projeto.

Referências externas

[1] ECMWF. ECMWF Reanalysis v5 ERA5. Descrição do produto, séries horárias, origem e resolução nativa. https://www.ecmwf.int/en/forecasts/dataset/ecmwf-reanalysis-v5

[2] Copernicus Climate Change Service. ERA5 hourly data on single levels from 1940 to present. Catálogo do conjunto de reanálise. https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels?tab=overview

[3] Open-Meteo. Historical Weather API. Modelo ERA5, grade de 0,25°, variáveis, unidades, seleção de célula e ajuste estatístico por altitude. https://open-meteo.com/en/docs/historical-weather-api

Data de consulta das referências de ERA5 e Open-Meteo: 18 de outubro de 2026. A aplicação usa o Open-Meteo como intermediário; não faz download direto do Climate Data Store.

Pontos do código usados como referência

Arquivo	Funções ou trechos centrais
app.py	Inicialização de main
painel.py	iniciar_base, recortar_base, mostrar_exposicao, mostrar_rotina, mostrar_persistencia, mostrar_saude, barras_agrupadas, main
dados.py	grade_horaria, limpar_horarios, interpretar_api, montar_conjunto, recortar_periodo, buscar_open_meteo
analise.py	indicadores_diarios, indicadores_noturnos, episodios_quentes, resumo_anual
saude.py	carregar_internacoes, serie_diaria, comparar_dias_quentes, comparar_por_faixas, comparar_todos
interpretacao.py	leitura_saude, leitura_faixas, conclusoes e os textos fixos da página
apoio.py e gerar_relatorio.py	numero e entrada alternativa de geração de relatório

Os nomes acima são relativos à raiz do projeto. Esta documentação registra o estado atual; mudanças futuras em filtros, cobertura, limiares, fontes ou funções devem ser refletidas no documento para preservar a reprodutibilidade.